



**KARADENİZ TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF SOFTWARE ENGINEERING**

**MINI LANGUAGE PARSER AND PARSE TREE
GENERATOR**

Student Name: Belkız Özmüş
Student Number: 439187

Date: May 10, 2026

Contents

1	Introduction	2
2	Aims and Objectives	2
3	Methodology	3
3.1	Test Results and Analysis	4
3.2	Parse Tree and JSON Output Visualization	4
3.3	Mathematical Model of Context-Free Grammar	5
3.4	System Features and Algorithm Steps	6
4	Findings	6
4.1	Error Detection and Reporting	6
5	Conclusion and Discussion	6
6	Recommendations	7

Abstract

This project aims to develop a syntactic analyzer that checks the validity of texts and performs their structural analysis according to defined Context-Free Grammar (CFG) rules. As the core method, the study employs a recursive descent algorithm that scans grammar rules in a hierarchical structure, along with tokenization strategy. The system automatically parses the input sentences into word or character level segments based on the structure of the terminals, and then tests the compliance of these segments with the grammar rules. The results are presented in two main formats: For successful analyses, data outputs in json format showing the sentence's hierarchical structure and visual analysis trees based on Graphviz are generated. In cases where the analysis fails, the system generates detailed error reports specifying the exact location of the error, the expected symbol category, and the cause of the error. As a result, this project offers a comprehensive tool that enables both the technical validation and visual representation of formal languages and grammatical structures.

Keywords

Context-Free Grammar, Syntax Parsing, Recursive Descent Algorithm, Syntax error detection

1 Introduction

This project focuses on developing a top-down, recursive descent parser designed to evaluate the syntactic correctness of sentences based on Context-Free Grammar (CFG). To adapt to the structural rules of various languages, the system is designed to be highly dynamic, featuring a robust BNF-style loader that reads grammar rules from external files and automatically adapts to tokenization at the word or character level, as well as seamless handling of epsilon transitions. For valid syntactic structures, the parser generates both a hierarchical JSON representation and automatic visual parse tree diagrams to clearly illustrate the derivation process. Conversely, to handle invalid sentences, the project implements a “reach-back” error reporting mechanism. By dynamically tracking the maximum index reached during backtracking, the system provides actionable, context-sensitive feedback, identifying exactly where the syntax failed and which specific tokens were expected. Ultimately, the core problem addressed by this project is: How can we programmatically evaluate whether a dynamically tokenized sequence conforms to a specially loaded CFG, and provide precise error diagnostics and clear visual representations for structural validation?

2 Aims and Objectives

Main Aim of the Project

The primary objective of this project is to develop a syntactic parser capable of parsing input text at the word or character level based on formal grammar rules dynamically loaded from an external source. The project aims to automatically check the compliance of given sentences with the specified grammar rules and to present the syntactic structure of valid sentences in JSON format and as visualizable Graphviz parse trees. Additionally, by

establishing a detailed error reporting mechanism that clearly indicates exactly where the error occurred in non-compliant sentences, which words were expected, and the structural cause of the error, the project aims to create a functional and educational analysis system that facilitates debugging in formal language and compiler design processes.

Project Objectives

To achieve the primary aim, the following sub-objectives have been established and implemented:

- **Rule Parsing:** Read CFG rules from external files and map them to efficient, hierarchical data structures.
- **Symbol Categorization:** Correctly distinguishing between terminal and non-terminal symbols within the provided grammar.
- **Tokenization:** Implementing a tokeniser that breaks down input sentences according to the defined terminal symbols.
- **Syntactic Analysis:** Verify sentence validity using a recursive descent parsing approach.
- **Visualisation:** Generate structured parse trees representing the derivation of valid sentences.
- **Error Handling:** Provide explanatory error messages that identify the exact location of syntactic errors and suggest expected symbols.

3 Methodology

In this study, a Syntactic Parser was developed to analyze input sentences based on user-defined Context-Free Grammar (CFG) rules. The process consists of three main stages:

1. **Data Preprocessing (Tokenization):** Grammar rules are loaded, and inputs are tokenized dynamically: character-level if all terminals are single characters, otherwise word-level. Empty string ϵ cases are also specifically handled.
2. **Parsing Algorithm:** The system uses Recursive Descent Parsing, applying a backtracking mechanism from the initial symbol to explore all possible derivation paths and verify grammar compliance.
3. **Error Handling and Output:** Successful parsing generates a hierarchical Parse Tree in JSON format and an auto-generated Graphviz URL for visualization. Parsing failures trigger a detailed console report pinpointing the exact error location, the unexpected symbol, and the list of expected symbols.

3.1 Test Results and Analysis

The parser was tested with various sentence structures to evaluate its accuracy in syntax validation and error detection. The results of these tests are summarized in Table below:

Input Sentence	Status	Error Reason
the man saw a dog	Valid	Correct syntax
the man the dog	Invalid	Unexpected token "the" at position 3 expected <verb>
liked the dog	Invalid	Unexpected token "liked" at position 1 expected <noun-phrase>
ϵ	Valid	Successfully handled via epsilon transition.
ba	Invalid	Unexpected token "a" at position 2 expected "b" or end of string.

Table 1: Parser Test Scenarios and Results

3.2 Parse Tree and JSON Output Visualization

In this section, the analysis results of the example sentence 'a telescope admired the cat' are presented to demonstrate how the system processes input. Figure 1 illustrates the Parse Tree of the sentence, while Figure 2 displays its corresponding JSON representation, which is used for structured data exchange.

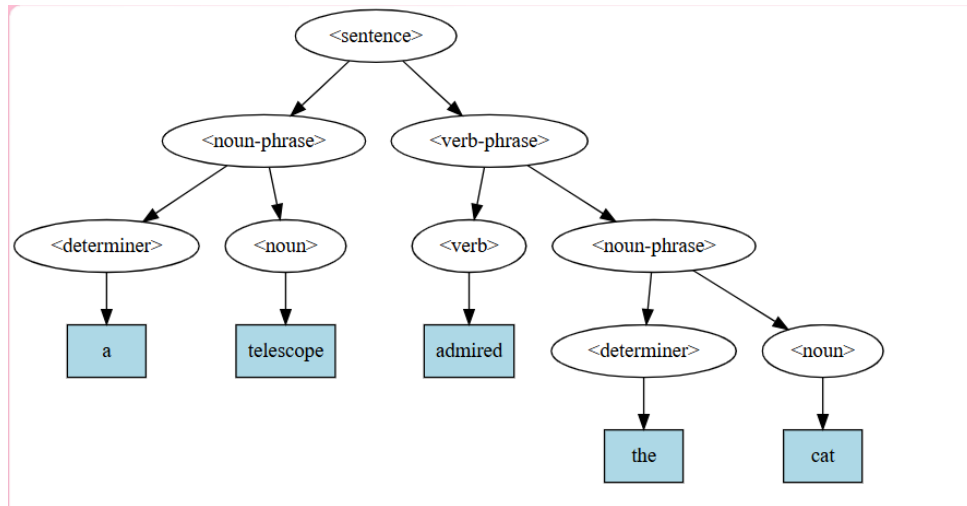


Figure 1: Parse tree representation of the sentence "a telescope admired the cat."

```

Input: a telescope admired the cat
Valid

Parse Tree:
Please click to see the tree

JSON:
{
  "sentence": {
    "noun-phrase": {
      "determiner": "a",
      "noun": "telescope"
    },
    "verb-phrase": {
      "verb": "admired",
      "noun-phrase": {
        "determiner": "the",
        "noun": "cat"
      }
    }
  }
}

```

Figure 2: JSON representation of the sentence "a telescope admired the cat."

3.3 Mathematical Model of Context-Free Grammar

A Context-Free Grammar (CFG) is mathematically defined as a 4-tuple:

$$G = (V, \Sigma, R, S)$$

where the components map directly to the implemented system as follows:

- V is a finite set of non-terminal symbols. In the Python implementation, these are represented by the keys within the `grammar` dictionary.
- Σ is a finite set of terminal symbols (the vocabulary of the language), which is dynamically extracted using the `get_terminals` function.
- R is a finite set of production rules. Each rule is mathematically formulated as:

$$A \rightarrow \alpha$$

where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$. In the codebase, these rules are stored as lists of valid symbol sequences for each non-terminal key.

- S is the start symbol ($S \in V$) that initiates the derivation and parsing process. The parser assigns the first parsed non-terminal rule from the file as this starting point (`start_symbol = list(grammar.keys())[0]`).

3.4 System Features and Algorithm Steps

This project includes several main features to process context-free grammars simply and effectively:

- **Grammar Loading:** Reads production rules directly from a text file
- **Tokenization:** Extracts words (terminals) and sorts them by length to read the input correctly
- **Parsing Algorithm:** Uses a recursive function to check if the input matches the grammar rules
- **Visual Output:** Creates a JSON tree and a web link to draw the parse tree automatically

The program works step-by-step as follows:

1. Loads the grammar rules and finds the starting point
2. Splits the input sentence into readable tokens
3. Checks the token list using the **Parser** function
4. If the sentence is correct, it generates the syntax tree in JSON and Graphviz formats
5. If the sentence is wrong, it shows the exact error location and what words were expected

4 Findings

The implemented top-down recursive parser was successfully tested with various sentences. The system showed strong performance in processing the Context-Free Grammar (CFG) rules. The findings are divided into two main areas: valid sentences and error detection.

4.1 Error Detection and Reporting

```
Input: liked the dog
Invalid
Error:
Where the error occurs: at token 1 liked
What was expected: a determiner ("a" or "the") to start
Why the sentence is invalid: the sentence begins with a verb, but grammar requires a determiner at the beginning
```

Figure 3: representation of the invalid sentence

5 Conclusion and Discussion

In conclusion, this project successfully developed a parser capable of evaluating sentences against Context-Free Grammar rules. The implementation effectively solved tokenization challenges by sorting terminal words by length, which prevented partial matching

errors. Through the recursive parsing algorithm, the system accurately distinguishes between syntactically valid and invalid inputs. For valid sentences, the program successfully generates structured JSON syntax trees and provides web links for visual Graphviz representations. For invalid sentences, the error-tracking mechanism provides clear and precise feedback by pinpointing the exact failure location and listing the expected words. Overall, the parser meets all the initial objectives. It provides a reliable and visual tool for analyzing grammar structures, successfully demonstrating the practical application of top-down parsing techniques.

6 Recommendations

Based on the current implementation of the parser, there are several areas for future improvement and research:

- **Handling Left Recursion:** The current top-down recursive descent algorithm may face infinite loops if the grammar contains left-recursive rules. A future update should include an automatic step to detect and eliminate left recursion before the parsing process begins.
- **Offline Visualization:** The current system generates a web link to display the syntax tree using the QuickChart API. Integrating a local visualization library would allow the program to draw and save parse trees completely offline.
- **Error Recovery:** Currently, the parser stops and reports only the first syntax error it encounters. Implementing an error recovery mechanism would allow the parser to skip the problematic word and continue checking the rest of the sentence for other potential errors.

Appendices

The architectural design of the developed system is structured to ensure a seamless transition from raw grammar rules to a validated syntax tree. Figure illustrates the high-level operational flow of the parser.

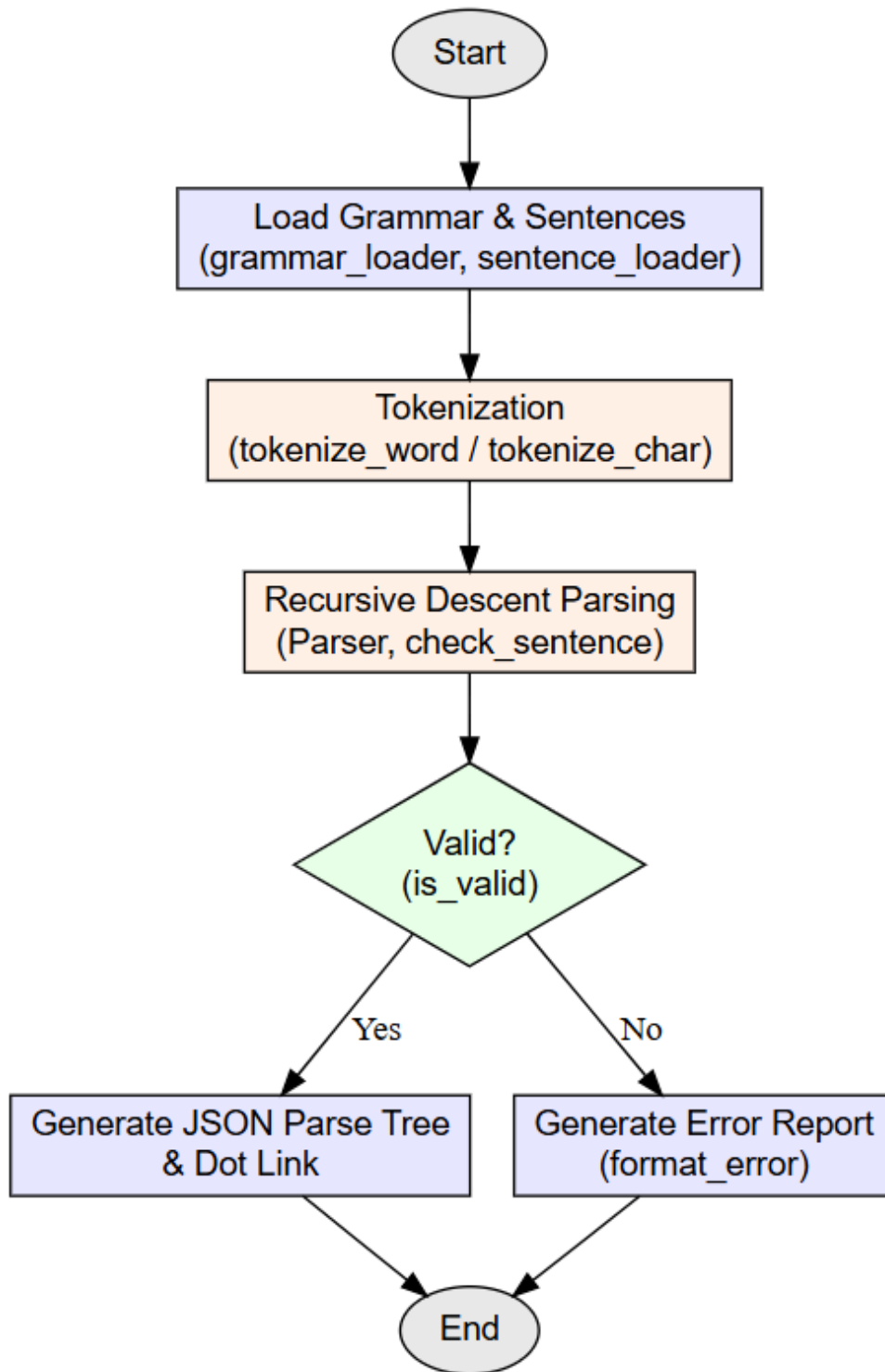


Figure 4: Operational flowchart of the Top-Down Recursive Descent Parser